

Make Lakehouse Catalogs Boring Again

Let's all agree on Iceberg's REST API and let the query engines dictate its evolution. Vendors can compete on enterprise features.



CHRIS

JUL 03, 2024



15



1



2

Share

There's plenty more to write about data lakehouse catalogs, but I'm going to call it with this post. My last write-up, below, explores where I'd like to see lakehouse catalogs go from here.

Before I get to that, there's a distinction to be made between lakehouse catalog use cases. I'm going to divide the use cases into "core" and "non-core" functionality. Core functionality provides query engines with the metadata they need to execute queries—the `information_schema` functionality that I talked about in [part 1 of this series](#). Non-core functionality is everything else that lakehouse catalogs do (or could do): user interfaces, data discovery, lineage, governance, and so on.

I would love to see everyone congregate around [Apache Iceberg's REST API](#) for core lakehouse catalog functionality. In fact, this is already happening. [Databricks's Unity](#) has (or will have?) Iceberg's REST API. Nessie [announced in May](#) that they, too, will have Iceberg REST endpoints. [Snowflake's Polaris will be an open source implementation](#) of Iceberg's REST API. [Many other Iceberg REST implementations](#) are popping up, too. [Gravitino, which I'm told](#) will soon begin incubating in Apache, looks particularly promising.

Adopting Iceberg's REST API is a great first step in cleaning up this mess. Next, I'd like to see query engines take a more active role in evolving the spec, particularly [Apache DataFusion](#) and [Trino](#). The core functionality that Iceberg provides is critical for query engines; they should be the ones dictating how it evolves.

Yes, this means that Databricks's [Spark](#) (and [Photon](#)) query engine will have some say. And yes, Snowflake is also a query engine that should have a say. But reorienting the API evolution around the query engines it serves means more projects can take an active role, thereby reducing the control of one or two vendors.

[I don't know exactly what an "active role" looks like](#) for these query engines. I would love for Trino and DataFusion to throw down the gauntlet and say, "This is the [OpenAPI](#) we're working with". DataFusion already has a [CatalogProvider](#) trait, it's just in Rust. Pushing one level deeper means query engines like Trino [will be able to deprecate Hive Metastore Service](#) (HMS), [AWS Glue](#) catalog, JDBC catalog, Nessie catalog, and Snowflake catalog integration. This will take time, but it'll be better in the long run.

Putting [Apache Arrow](#) and [Apache DataFusion](#) in the driver's seat is something that [LanceDB](#) did with [Lance V2](#). Rather than define their own types and encodings, they're deferring to Arrow—something I wrote about in [Nimble and Lance: The Parquet Killers](#). Applying a similar approach in the catalog space seems wise to me.



Chris Riccomini

@criccomini

...

Crazy thought: should the [@ApacheDataFusio](#) build a catalog? The space is kinda a mess right now. A nice clean Apache catalog might be just what the doc ordered.

[View Post](#)



enterprises. In fact, Databricks's (non-open source) [Only page](#) focuses entirely on such features—discoverability, permission management, governance, monitoring, observability, and so on. This is where the vendors should compete.

From this vantage point, it seems data lakehouse vendors have accidentally stumbled upon the wedge that traditional data catalogs have been struggling to find. Traditional data catalogs have been unable to shift from nice-to-have to must-have in the enterprise; they remain a dusty tool that's often ignored once installed. Data lakehouse catalogs are must-have for anyone running query engines on a data lake. From there, eating into the customer base of [Alation](#), [Datahub](#), and others seems natural.

Other posts in this series are available here:



Begun, The Catalog Wars Have

CHRIS RICCOMINI • JUNE 20, 2024

[Read full story](#) →



Data Lakehouse Catalog Reality Check

CHRIS RICCOMINI • JUNE 27, 2024

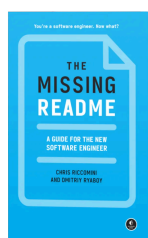
[Read full story](#) →

Type your email...

Share

Book

Support this newsletter by purchasing [The Missing README: A Guide for the New Software Engineer](#) for yourself or gifting it to someone.



Buy Now



Discover more from Materialized View

Software infrastructure hot takes, projects, papers, developer interviews, and deep dives. Brought to you by Chris Ricchini.

Over 6,000 subscribers

Enter your email...

Subscribe

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).

Already have an account? [Sign in](#)

Disclaimer

I occasionally invest in infrastructure startups. Companies that I've invested in are marked with a 📌 in this newsletter. See my [LinkedIn profile](#) for a complete list.



15 Likes • 2 Restacks



← Previous

Next →

Discussion about this post

Comments Restacks

Commenting has been turned off for this post



Kevin Liu

Jul 3, 2024

...

♥ Liked by Chris

> Adopting Iceberg’s REST API is a great first step in cleaning up this mess.
□□□ Hopefully by this time next year, everyone will have already implemented all the core functionalities, and we'll just be fighting over the non-core ones.

♡ LIKE (1)

↑ SHARE

S3 Is Showing Its Age

I'm squarely in the trough of disillusionment with S3.

MAY 22, 2024 • CHRIS

♡ 35

↻ 6

↑



Kafka: The End of the Beginning

A decade of focus on adoption has paid off. Now it's time to innovate.

MAY 30, 2025 • CHRIS

♡ 34

↻ 4

↑



It's Time to Merge Analytics and Data Engineering (Again)

Data pipelines are commoditized and analytics engineers don't provide enough value.

NOV 18, 2024 • CHRIS

♡ 45

↻ 3

↑



See all >

Ready for more?

Type your email...

Subscribe



 **Start your Substack**

Get the app

Substack is the home for great culture